Index construction and MapReduce

Hans Philippi
(partially based on the slides from the Stanford course on IR)

April 24, 2026

The role of postings lists

Given a document collection, a *Postings List* for a search term t is a sorted list of document id's, where these documents contain term t

Query = $term_1$ AND $term_2$

1. locate postings list p_1 for $term_1$
2. locate postings list p_2 for $term_2$
3. calculate the intersection of p_1 and p_2 by list merging

$term_1 \Longrightarrow$	1	3	7	**11**	37	**44**	**58**	112	...
$term_2 \Longrightarrow$	2	4	**11**	25	**44**	54	55	**58**	...

Index for text collections

INPUT: a term
OUTPUT: the corresponding postings list, i.e. all occurences of this term in the document collection containing this term.

Two steps:

- Use a tree structure (B-tree, suffix tree) that connects a term to the corresponding postings list
- Return the postings list

Phrase queries

Make a distinction between:

Q1 = "fight" AND "club"

Q2 = "fight club"

How do we support juxtaposition of terms?

Phrase queries

How do we support juxtaposition of terms?

Solution 1: biword index

Disadvantages:

- index size quadratic
- how do we support juxtaposition of three or more terms?

Phrase queries

How do we support juxtaposition of terms?

Solution 1: biword index

Disadvantages:

- index size quadratic
- how do we support juxtaposition of three or more terms?

Solution 2: positional index

Positional index

For each term, we also register the position(s) of the term in each document, where a document is regarded to be an array of tokens.

So, for each term *myterm*, we have the following entry in the index:

< *myterm*: nr of docs containing *myterm*;
    doc1: position1, position2, ... ;
    doc2: position1, position2, ... ;
    ...
>

Positional index

Example:

<*be*: 993427;
1: 7, 18, 33, 72, 86, 231;
2: 3, 149;
4: 17, 191, 291, 430, 434;
5: 363, 367;
... >

Which of the docs could contain:
"to be or not to be"

Index construction: the main task

- INPUT: the document collection
- OUTPUT: the postings lists for all terms in the document collection
- When we have the postings lists, the tree structure can be created based on the lexicographic ordering of the terms; this is a secondary task
- Classic algorithms deal with limited main memory, based on external sorting
- Alternative: index building based on MapReduce; generic architecture for and approach to large scale parallellism
- Applying MapReduce to this problem is our goal for today!

MapReduce: an example of NoSQL data management

- Data model: key-value pairs
- Roots in Google environment (indexing, PageRank)
- MapReduce facilitates massive parallellism on large amounts of commodity hardware
- It provides the programmer with a generic interface for parallel programming ...
- ... and supports transparency with respect to low level implementation issues, especially robustness

MapReduce

- Generic, based on Map and Reduce (Fold) from functional programming
- Two sets of machines involved in parallel processing: Map workers and Reduce workers
- Robustness: failure of workers covered by duplication
- Several implementations, Hadoop is the most well known

Creating postings lists: an example

Input : document collection consisting of pairs <docid, text>

$<$ 2013, "de dag die je wist dat zou komen is eindelijk hier"$>$
$<$ 1980, "de do do do, de da da da"$>$
$<$ 1971, "jaren komen en jaren gaan"$>$
$<$ 1994, "we komen en we gaan"$>$

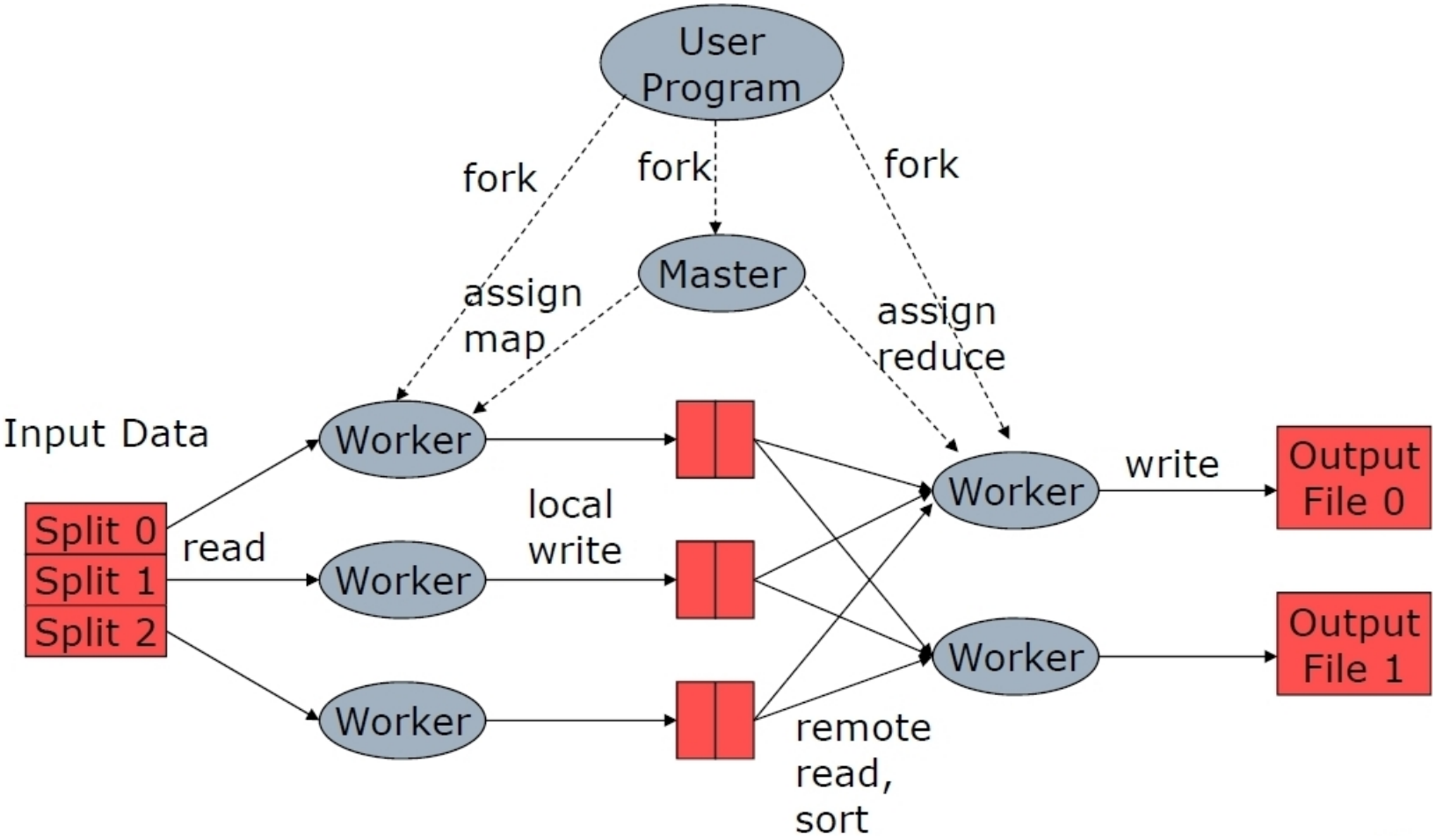
Output : a set of *postings lists* for this collection of documents

...
$<$"dag", [2013] $>$
$<$"de", [1980, 2013] $>$
$<$"die", [2013] $>$
$<$"do", [1980] $>$
$<$"en", [1971, 1994] $>$
$<$"gaan", [1971, 1994] $>$
...
$<$"komen", [1971, 1994, 2013] $>$
...

MapReduce: the Map

- Basic data structure is key-value pair $< k, v >$
- Input is split into disjoint chunks, containing collections of key value pairs
- Each Map worker works autonomous from other map workers ("shared nothing")
- Each Map worker scans it's own input chunk once
- Each Map worker does one uniform calculation on each key-value pair
- The output of each Map worker is a set of key-value pairs: zero, one or more
- The structure of the resulting key-value pairs is generally different from the input pairs
- The output results of all Map workers are collected for further processing in the Reduce phase

MapReduce computing



MapReduce: the Reduce

- The output results of all Map workers are grouped on the key values and assigned to the reduce workers
- All related key value pairs will be processed by one Reduce worker
- Each Reduce worker works autonomous from other Reduce workers (shared nothing)
- The output results of all Reduce workers together are the result of the calculation

MapReduce example

Example: *word count*

Input: a collection of documents
Output: the words in the documents with their frequency

- Map $< docid, text >$:
  for each word w in $text$
  $emit(< w, 1 >)$;
- Reduce $< w, vlist >$:
  ...

MapReduce example

Input to Map-workers:

< 2013, "de dag die je wist dat zou komen is eindelijk hier" >
< 1980, "de do do do, de da da da" >
< 1971, "jaren komen en jaren gaan" >
< 1994, "we komen en we gaan" >

Output from Map workers:

<"de", 1 >
<"dag", 1 >
<"die", 1 >
...
<"gaan", 1 >

MapReduce example

... then comes the invisible step ...

... which could be characterized as a "GROUP BY key" ...

MapReduce example

Input to Reduce-workers:

$<$"de", $[1]>$

...

$<$"komen", $[1, 1, 1]>$

...

$<$"gaan", $[1, 1]>$

...

Output:

$<$"de", $1>$

...

$<$"komen", $3>$

...

$<$"gaan", $2>$

...

MapReduce example

Example: *word count*

Input: a collection of documents
Output: the words in the documents with their frequency

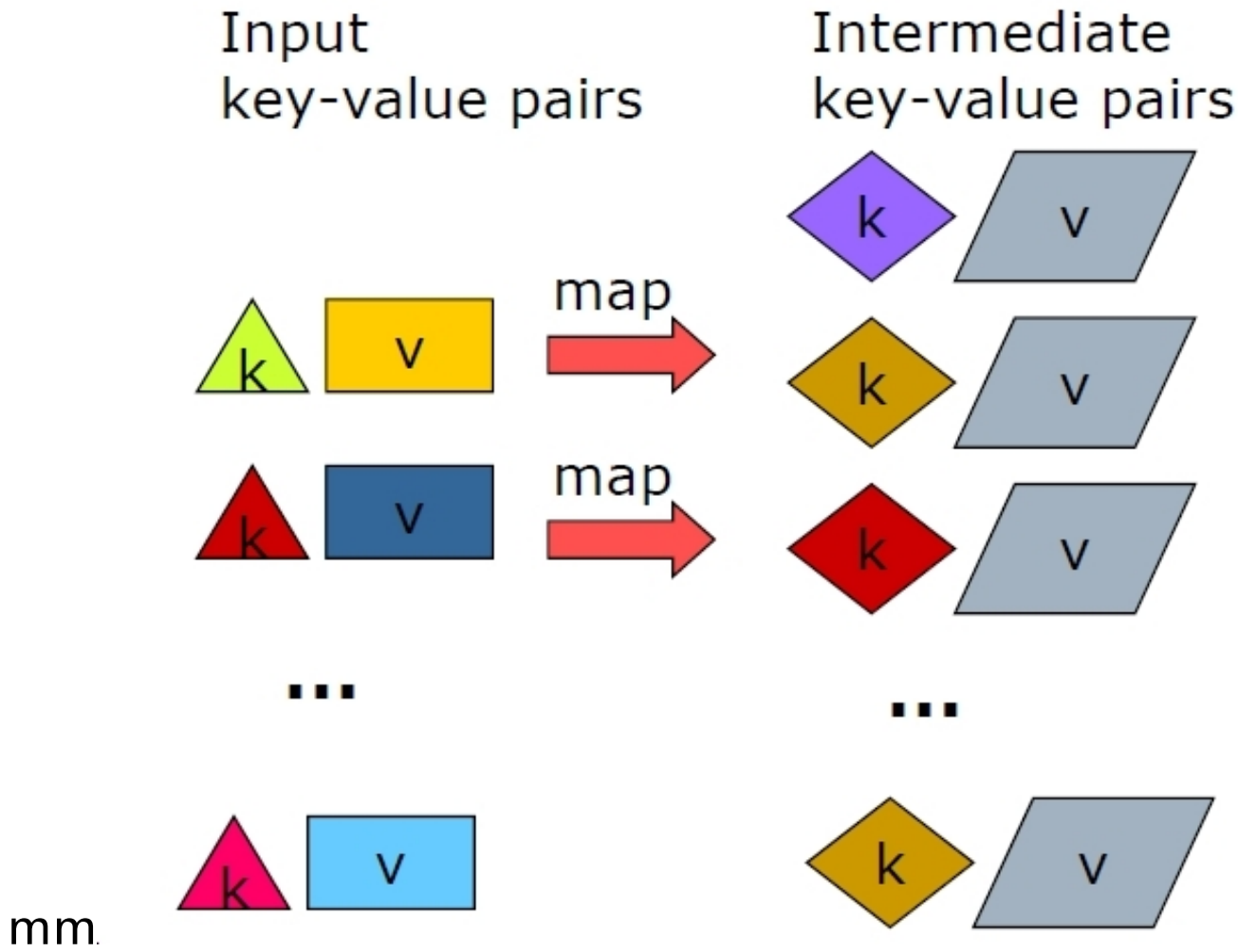
- Map $< docid, text >$:
  for each word w in $text$
    $emit(< w, 1 >)$;
- Reduce $< w, vlist >$:
  int $sum = 0$;
  for each v in $vlist$
    $sum++$;
  $emit(< w, sum >)$;

MapReduce example

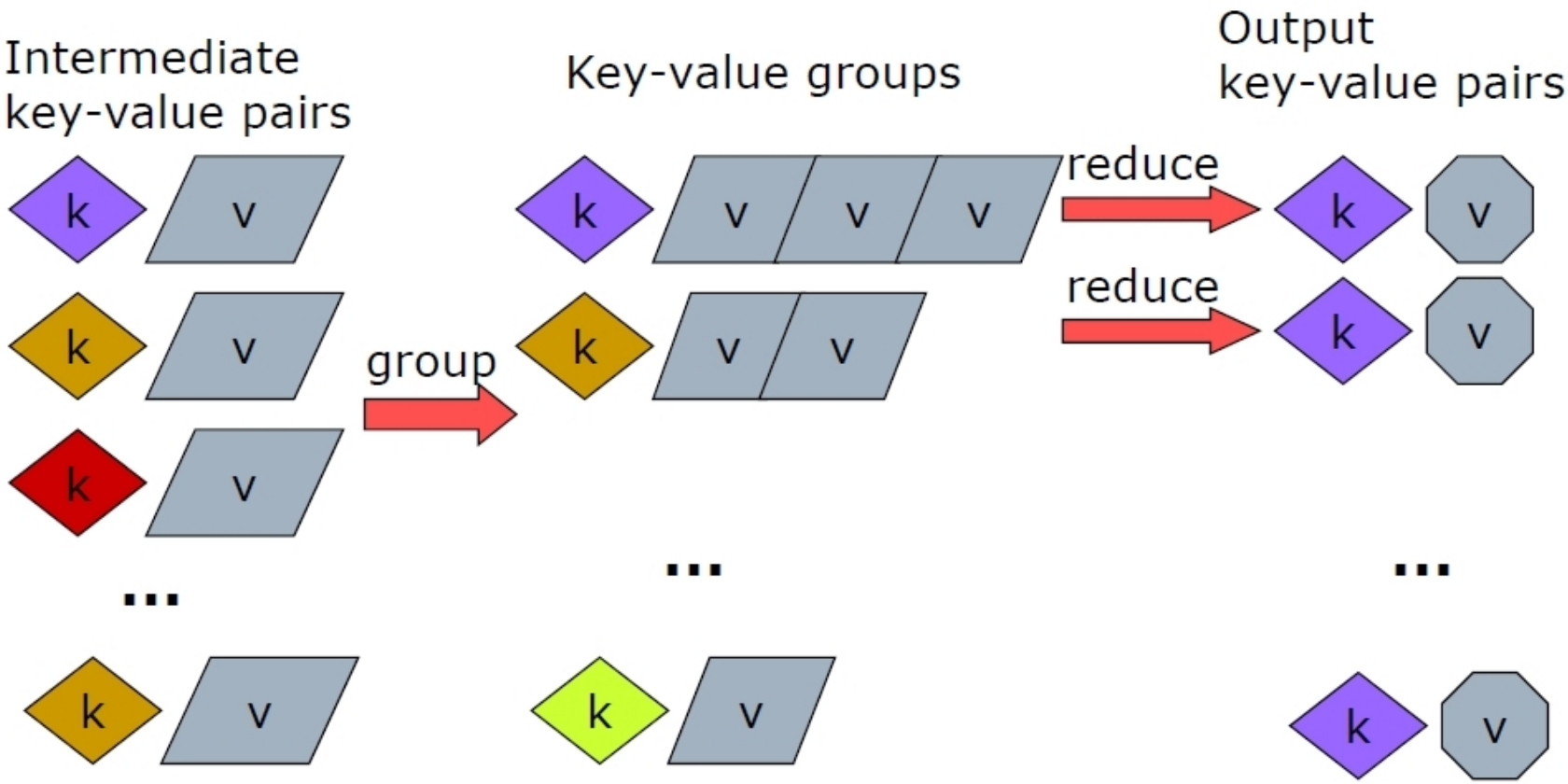
Observations:

- The input pairs will be processed by different Map-workers
- Behind the scenes (invisible step), all emitted pairs with the same key are grouped together (after the Map phase and before the Reduce phase)
- The *grouping phase* includes concatenation of all values corresponding to the same key
- In our example: in the grouping phase: three times $<$"komen", $1>$ becomes $<$"komen", $[1, 1, 1]>$

MapReduce computing: the Map



MapReduce computing: the Reduce



INTERMEZZO MapReduce example

Do you have any suggestions for optimization of the MapReduce program from the example on slide 15?

MapReduce computing: early combining

- Word count could be optimized by doing some aggregation in the Map phase
- Instead of k repetitions of $emit(< w, 1 >)$; do $emit(< w, k >)$;
- Adapt the Reduce program (how?)
- In general, this idea is applicable if the reduce function is commutative and associative (e.g. sum, max)
- Early combining often requires a setup of local datastructures and a final emit
- Our convention: for writing pseudo code, use functions *Init_Map*() and *Finalize_Map*()

Word count speed up

- Init_Map():
  Create a dictionary D (word, freq);
- Map $< docid, text >$:
  for each word w in $text$
  add w to D;
- Finalize_Map():
  for each entry (word, freq) in D
  $emit(< word, freq >)$;
- Reduce $< w, vlist >$:
  int $sum = 0$;
  for each v in $vlist$
  $sum+ = v$;
  $emit(< w, sum >)$;

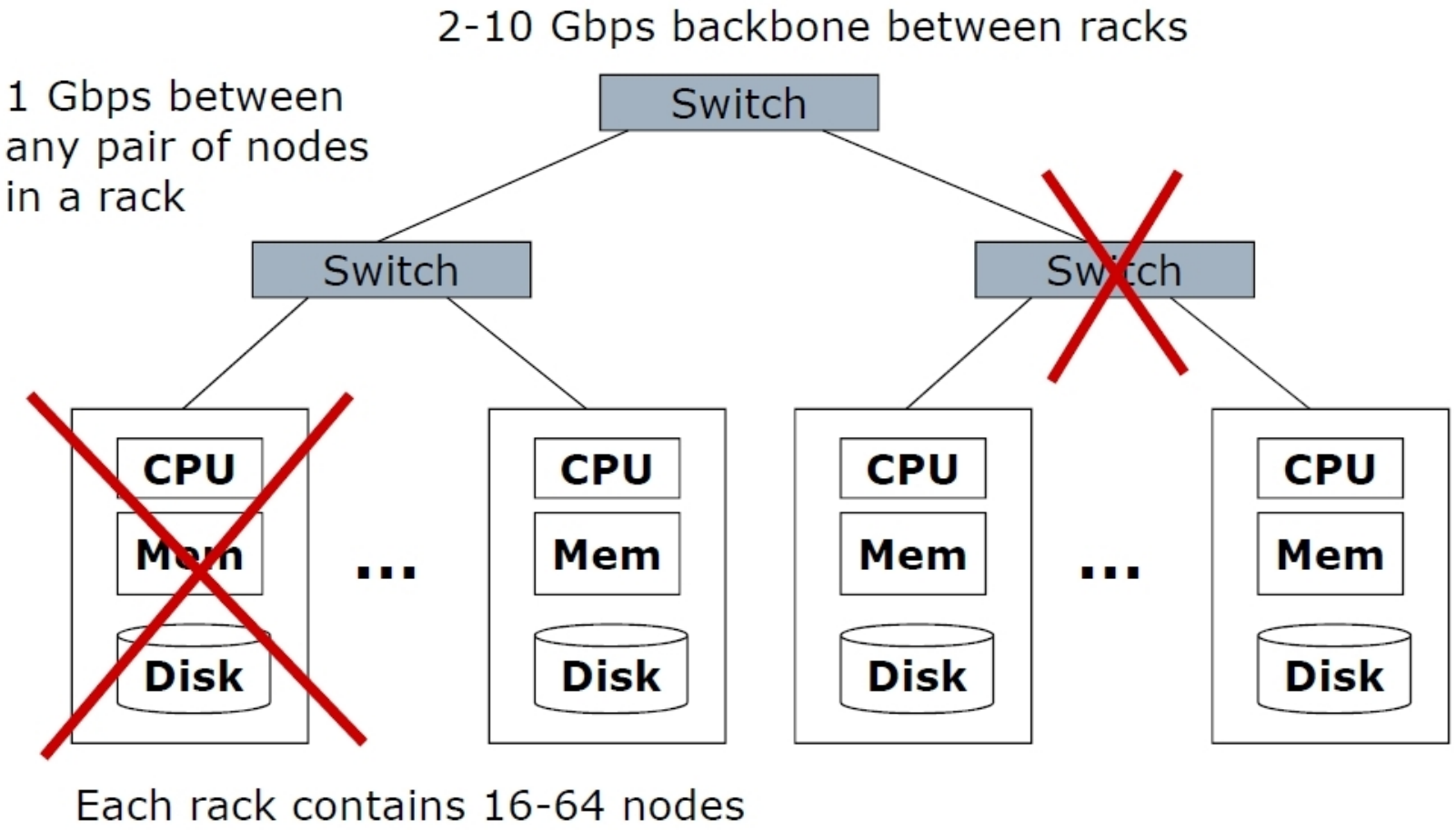
The rules of the game!

- Neither Map nor Reduce have explicit access to their input stream
- Both consist of a generic piece of code that is repeated for each <key, value> pair in the input stream (except for Init and Finalize)
- This repetition is not part of the Map or Reduce code, but is done behind the scenes
- So do **not** loop over the input file!
- But in the Reduce, the *value* is a list itself, so in general you write a loop for it
- The only way to generate output is the "emit(<k,v>)" command
- You can only emit textual data (characters, numbers, lists, records); you cannot emit binary structures like dictionaries

MapReduce architecture (original paper)

- We are dealing with terabyte scale processing problems
- Standard shared-nothing architecture
- Cluster of commodity Linux nodes
- Gigabit ethernet interconnection
- cheaper than supercomputer
- Masking hardware failures
- Input and final output on distributed file system

MapReduce architecture (original paper)



MapReduce architecture: DFS (original paper)

- Distributed file system for input and output
- Terabyte scale
- Data warehouse behaviour
- Read intensive, rarely updated
- File is split into large contiguous chunks
- Data 2-3 times replicated in different racks

MapReduce computing: coordination

Master process

- determines number of Map tasks (M) and number of Reduce tasks (R)
- M and R are chosen much larger than the number of nodes
- monitors status workers: *idle*, *busy*, *down*
- monitors status tasks: *idle*, *in-progress*, *completed*

After finishing, a Map task delivers R intermediate result files on the local disks of the Map workers and sends the sizes and locations to the Master

- Master forwards this info to the reduce workers

MapReduce computing: partitioning

- The intermediate key-value pairs with the same key must end up at the same Reduce worker
- System uses a default partition function to split up Map output: hash(key) mod R
- It is possible to override the default partition function

MapReduce: references

- http://infolab.stanford.edu/~ullman/mmds/ch2.pdf